COMPLETE DOCUMENTATION AND SOURCE CODE

Schedule Assistant

A private meeting and task scheduler that lives on your own computer - with a Gmail invitation inbox, Google Calendar sync, an auto-updating Drive worksheet, and a reminder 30 minutes before every event.

Creator: Dr. Asif Ahmad

Version: 2.0 - WhatsApp control, present in the first build, was removed in this version at the owner's request.

Document: Overview, applications, how it works, workflow, a beginner-friendly how-to-use guide, features, benefits, limitations, uniqueness, the security model, deployment and operating-system guides, glossary and troubleshooting.

Appendix A: The complete source code - server, core store, Sheets sync, web page, frontend logic, package file, PWA manifest and start scripts - reproduced verbatim with line numbers, so the app can be rebuilt from this document alone.

The app's address on your computer: http://127.0.0.1:4318 - it is a local address, not a public website. The app runs only while you start it on your own machine, which is exactly what keeps it private.

Tests: 15 automated tests pass. They use temporary folders and never send real emails or touch real Google accounts.

CONTENTS

What this document covers

1. Overview - what the Schedule Assistant is 3

2. Applications - where it is used 4

3. How it works - architecture and the four engines 5

4. Workflow - from invitation to reminder 7

5. How to use - setup, phone access and daily use 8

6. Features - the full table 10

7 and 9. Benefits - and what makes it unique 11

8 and 10. Limitations - and the security model 12

11 and 12. Deployment and operating systems 13

13. The project files 14

Appendix A - Complete source code (746 lines, verbatim) 15

Appendix B - Glossary and troubleshooting FAQ 39

About this document

This is the complete record of the Schedule Assistant: what it is for, how its four engines work, how to set it up and use it day by day with no technical background assumed, where its limits honestly lie, and the full code that makes it run. Nothing in the app is a black box - its entire logic is printed in Appendix A with line numbers, and the app could be rebuilt from Appendix A alone.

The app's link

```
http://127.0.0.1:4318
```

That address is the app. It opens only on the computer running the app (or, in phone mode, on your own Wi-Fi). There is no public web link and no account to create - privacy is the design, not a setting.

> **Creator credit.** The Schedule Assistant was designed and generated by **Dr. Asif Ahmad**. This documentation is credited to him throughout, on the cover and in every page footer, matching the credit line inside the app itself.

Conventions: times are Pakistan time (Asia/Karachi) unless storage is being explained; file names appear in code style and app page names in bold.

SECTION 1

Overview - what the Schedule Assistant is

The Schedule Assistant is a private meeting, appointment and task scheduler that runs on your own computer. It keeps your records in a database file on your disk, watches your Gmail for invitations, syncs with Google Calendar, maintains a live worksheet in your Google Drive, and emails you a reminder 30 minutes before every event.

Most scheduling tools are cloud services: your meetings sit on a company's servers, behind a subscription. This app takes the opposite approach - a small Node.js program on your laptop serving a web page to your own browser, hosted nowhere else. The only outside service it talks to is your own Google account, and only after you connect it yourself with one sign-in click.

What it does, in one list

- **Records** meetings, appointments and tasks - create, edit, cancel, complete, search and export them.
- **Invitation inbox:** every five minutes it scans Gmail for messages that mention meetings or appointments and files them as suggestions, each with a parsed date and time. Nothing is scheduled until you approve it.
- **Google Calendar sync:** two-way - existing timed events come in, and any event can be pushed out with Save to Google, gaining Google's own 30-minute reminders.
- **Drive worksheet:** every entry is written automatically as a row in a Google Sheets workbook called "Asif Schedule Assistant", with a Dashboard tab of live counts.
- **Reminders:** a background check runs every 15 seconds. Thirty minutes before an event, you receive an email reminder and a dashboard entry, and the delivery is logged in a history you can inspect and retry. On the same Wi-Fi, the app also installs on an Android phone as a home-screen icon (a Progressive Web App).

The facts

Item	Detail
Creator	Dr. Asif Ahmad
Version	2.0 (WhatsApp control removed per owner request)
Default account	asifahmad1@gmail.com
Address on your computer	http://127.0.0.1:4318
Runs on	Node.js 24 or newer, on Windows, macOS or Linux; phone access via Android PWA
Records stored	SQLite database in a private folder on your own computer
Times	Stored in UTC; always displayed in Pakistan time (Asia/Karachi)
Cost	Free. No subscription or per-user fees.

> **Version 2 note.** Version 1 could be controlled by WhatsApp messages; that feature was removed completely in version 2.0 at the owner's request. This document describes version 2.0 only.

SECTION 2

Applications - where it is used

The app was built for one person's working life: a busy academic who teaches, supervises, sits on committees, reviews papers and applies for positions - and whose invitations arrive by email.

Personal meeting scheduling

Faculty meetings, committee sittings, viva examinations, supervision sessions and calls with collaborators are created in seconds with a title, date, time, duration, location and notes. The week view shows all seven days at a glance, colour-coded by type, and clicking any day opens the new-event form for that date.

Appointment and task tracking

Three event types keep different kinds of commitments apart: meetings (green), tasks (orange) and appointments (purple). Each record carries a status - scheduled, completed or cancelled - so a deadline for a manuscript review or a hospital appointment is tracked to done, not just remembered.

Reviewing Gmail invitations

Invitations that arrive by email no longer have to be copied into a calendar by hand. The invitation inbox collects them, suggests a date and time parsed from the message text, and waits. One click accepts the suggestion as an event; another dismisses it. A message can also be pasted into the inbox manually when it arrived by another route.

Calendar consolidation

Events already sitting in Google Calendar are imported, and anything created in the app can be saved back to Google. The app and the calendar stop being two separate lists that drift apart.

Reminder automation

Thirty minutes before every event, an email reminder arrives - enough time to find a room, open a file or join a call - whether or not the app's page is open in the browser. The reminder history shows what was delivered and what was not, so a missed reminder is visible rather than silent.

Worksheet reporting

The auto-updating Drive worksheet doubles as a log and a report: every commitment as a spreadsheet row, with live counts on the Dashboard tab. It can be opened, shared or printed from Google Drive like any other spreadsheet - useful for a weekly review or an annual record of meetings attended.

> **One app, one owner.** This is a single-user tool by design. It is not a team calendar and does not try to be one; its value is that one person's schedule is complete, private and accounted for.

SECTION 3

How it works - the architecture

Three program files do the server-side work, and three files make the page you see. Understanding which file does what is the whole architecture.

File	What it does
`server.mjs`	The Node.js HTTP server. Serves the web page and a JSON API, handles Google sign-in, scans Gmail, syncs Calendar, runs the reminder engine, and answers every button you press.
`core.mjs`	The store. Opens the SQLite database, validates every event, encrypts secrets with AES-256-GCM, and works out which reminders are due.
`sheets.mjs`	The worksheet sync. Creates and maintains the "Asif Schedule Assistant" spreadsheet in Google Drive - Schedule tab, Dashboard tab, status colours, live count formulas.
`public/index.html`	The page structure: sidebar, six views (Overview, My schedule, Invitation inbox, Connections, Reminder history, Settings) and the event form.
`public/app.js`	The frontend logic: loads records from the API, draws the week calendar and agenda, and sends every create, edit and approval back to the server.
`public/style.css` and `manifest.webmanifest`	The appearance, and the file that lets the app install on a phone home screen as a PWA.

The store: SQLite with encrypted secrets

All records - events, inbox items, reminder deliveries, settings - live in one SQLite database file inside a private data folder on your computer (Section 11 gives the exact locations). SQLite is a real database with no server of its own: the file survives browser closures and restarts, and copying it copies the whole record.

Google tokens and the OAuth client secret are never stored in plain text. They are sealed with AES-256-GCM encryption using a key kept in a `local.key` file in the same protected folder, with owner-only file permissions. The export button exports records only - never tokens or credentials.

Validation, before anything is saved

Every event passes the checks in `core.mjs` before it is stored: a title of 1 to 180 characters, a type of meeting, task or appointment, a valid start and end with the end after the start, and a maximum duration of seven days. Times are stored in UTC and converted to Pakistan time for display, so nothing shifts if the computer's timezone settings change.

Google sign-in: OAuth2 with PKCE

The connection uses Google's OAuth 2.0 flow for desktop apps, hardened with PKCE (an S256 code challenge), and the sign-in returns to a loopback address on your own machine only. You create a Desktop OAuth client once in Google Cloud Console, upload its JSON file on the Connections page, and click Sign in with Google once. The app then keeps the encrypted refresh token, so it stays connected across restarts without asking again.

How it works - the four engines

1. The invitation scanner (every 5 minutes)

Once Google is connected, the server searches Gmail every five minutes for recent messages that mention meetings or appointments. Each new message becomes an inbox item. The `chrono-node` library parses English date expressions in the text - "next Tuesday at 3 pm", "on 12 March" - and proposes a start and end time with a confidence note.

> **Approval first, always.** A suggestion is never scheduled on its own. English date parsing is good but not perfect, so every suggestion waits in the inbox for your Accept or Dismiss. That single rule is what makes the inbox safe to trust.

2. Calendar two-way sync

On the same five-minute cycle, timed events are imported from Google Calendar. In the other direction, the Save to Google button on any event creates the matching Google Calendar event - and because Google attaches its own 30-minute email and popup reminders, that event keeps reminding you even if this app is later closed.

3. The worksheet sync (4-second debounce)

Any change - a new event, an edit, a completion - schedules a worksheet update. The sync waits four seconds after the last change (a debounce), so a burst of edits becomes one update, then rewrites the Schedule tab with one row per event, times shown in Pakistan time, with status colouring, and stamps the Dashboard tab's "Last synced" cell. The Dashboard's counts are live spreadsheet formulas, so they are always current. The workbook is created automatically on first connection; its ID is stored locally so every later sync reuses the same spreadsheet instead of making copies.

4. The reminder engine (every 15 seconds)

A timer wakes every 15 seconds and asks the store which scheduled events start within the next 30 minutes. For each one, a reminder is due on two channels: an email to yourself, sent through Gmail, and a dashboard entry. Each delivery is claimed once - the database records the claim before sending - so the same reminder is never sent twice.

Every attempt ends with a receipt in the reminder history: **sent** (Gmail accepted it), **uncertain** (the send could not be confirmed), or **dashboard**. Uncertain deliveries are never replayed automatically - replaying could duplicate an email that did in fact arrive. Instead, the Reminder history page offers an explicit Retry button, so the decision to send again is yours.

> **Late is honest.** If the computer was asleep or offline when a reminder fell due, it is sent when the app next runs, and its text states the actual minutes remaining. No reminder is sent after an event has already started.

SECTION 4

Workflow - from invitation to reminder

Every event follows the same path: created or approved by you, stored, optionally saved to Google, mirrored to the worksheet, reminded, and logged.

1. **An event starts in one of two ways**
   You create it on the Overview or My schedule page - or a Gmail invitation lands in the inbox with a suggested date and time.

| v

2. **You approve anything the app suggested**
   Inbox suggestions wait. Accept turns one into a real event; Dismiss removes it. Events you type yourself skip this step.

| v

3. **The event is validated and stored**
   Title, type, start, end and duration pass the checks in core.mjs, and the record is written to the SQLite database.

| v

4. **Save to Google (your choice, per event)**
   The event is created in Google Calendar, gaining Google's own 30-minute reminders as a second safety net.

| v

5. **The worksheet updates itself**
   Four seconds after the last change, the Schedule tab gains the new row and the Dashboard counts move.

| v

6. **Thirty minutes before: the reminder**
   The 15-second reminder tick finds the event and delivers the reminder by email and on the dashboard, once per channel.

| v

7. **The delivery is logged**
   Reminder history records sent, uncertain or dashboard for every reminder. Uncertain ones wait there with a Retry button.

| v

8. **You complete or cancel it**
   After the event, mark it Completed, or Cancel it if it falls through. The worksheet updates again.

Two background cycles keep this moving: the five-minute Gmail and Calendar cycle, and the fifteen-second reminder tick.

SECTION 5

How to use - first-time setup

No programming knowledge is needed. If you can install a program and double-click a file, you can set this app up. Allow about fifteen minutes, most of it for the one-time Google connection.

Step 1 - Install Node.js

Go to `https://nodejs.org` and install the **Node.js 24 LTS** version (the app needs Node 24 or newer). Google Chrome is the recommended browser.

Step 2 - Put the app folder somewhere permanent

Extract the app zip into a folder you will keep - for example inside Documents. Do not run it from inside the zip or from the Downloads folder.

Step 3 - Start the app

On Windows, double-click **Start-Windows.cmd**. The first run installs the app's two libraries automatically and may take a minute; later starts are quick. On macOS or Linux, run `bash start.sh` in the folder instead. Leave the small black window open while you use the app - closing it stops the app.

Step 4 - Open the app

```
http://127.0.0.1:4318
```

The browser usually opens by itself. If it does not, type that address into Chrome exactly as shown. This is the app's link: it works only on your own computer, which is the point - there is no public website and no password, because no one else can reach it.

Step 5 - Connect Google (one time only)

1. In Google Cloud Console, create a project and a **Desktop app** OAuth client.
2. Enable four APIs for the project: **Gmail, Google Calendar, Google Drive and Google Sheets**.
3. Download the client's JSON file.
4. In the app, open **Connections**, upload that JSON file, then click **Sign in with Google**.
5. Choose **asifahmad1@gmail.com** and approve the consent screen - this one click is the only sign-in the app will ever ask for.

The folder also contains `EASY-SETUP.md`, a beginner guide that walks through the Google steps with more detail, and `SETUP.md`, the technical reference.

> **What happens after connecting:** the Gmail scan and Calendar import begin on their five-minute cycle, and the "Asif Schedule Assistant" worksheet is created in your Drive automatically the first time an event syncs.

How to use - phone access and daily use

Put it on your phone (Android)

1. On the laptop, double-click **Start-Phone-Access.cmd** instead of the normal start file. It starts the app in LAN mode and prints the address your phone should use.
2. Connect the phone to the **same Wi-Fi** as the laptop and open that printed address in Chrome on the phone.
3. Tap the browser menu and choose **Add to Home screen**. The app installs as an icon - "Schedule" - and from then on opens full-screen like a normal app.

Phone mode accepts only your own local network addresses. It is for your home or office Wi-Fi, never for exposing the app to the internet.

Daily use: the six pages

Page	What you do there
Overview	Your day at a glance: today's count, this week's count, invitations waiting, your next event, the connection status, and the seven-day week calendar. Click any day to add an event on it.
My schedule	Every record in one agenda list. Search by words, filter by upcoming, all, completed or cancelled. Each row can be edited, completed, cancelled, or saved to Google.
Invitation inbox	Review what Gmail found. Each card shows the message and the suggested time; Accept schedules it, Dismiss clears it. You can also paste an invitation in by hand.
Connections	Google connection status, a Sync now button, the link to your Drive worksheet, and the app download. This is also where the one-time Google setup lives.
Reminder history	Every reminder the app has delivered, with its receipt - sent, uncertain or dashboard. Unconfirmed deliveries carry a Retry button.
Settings	Your email address, reminder on/off switches, the automatic sync toggle, and the Export button that downloads a JSON backup of all your records.

Creating an event, in ten seconds

1. Click **New event** (top right) or click a day in the week calendar.
2. Type the title, pick meeting, task or appointment, set the date, start time and duration, and add a location or notes if you wish.
3. Save. The event appears in the agenda and the week view, syncs to the worksheet within seconds, and its 30-minute reminder is armed.
4. Optional: click **Save to Google** on the event to add it to Google Calendar as well.

> **Footer credit.** Every page of the app carries the line "Created by Dr. Asif Ahmad" in its footer - the app is signed by its creator, like this document.

SECTION 6

Features - the full table

Feature	Detail
Events	Create, edit, cancel and complete meetings, tasks and appointments, with title, date, time, duration, location and notes. Titles up to 180 characters; events up to seven days long.
Week view	Seven-day calendar on the Overview page, colour-coded by type; click a day to create an event on it.
Search and filter	Full-text search plus upcoming / all / completed / cancelled filters on the My schedule page.
Invitation inbox	Gmail scanned every five minutes; each invitation arrives with a chrono-node date suggestion and a confidence note; approval-first scheduling; manual paste also supported.
Google Calendar	Two-way sync: timed events imported automatically; Save to Google exports any event and attaches Google's own 30-minute reminders.
Drive worksheet	Auto-created "Asif Schedule Assistant" spreadsheet: Schedule tab (one row per event, Pakistan times, status colours) and Dashboard tab (live count formulas, last-synced stamp). Updates within seconds, debounced at four seconds.
Reminders	Email plus dashboard at 30 minutes before, checked every 15 seconds, delivered once per channel, with honest late text showing actual minutes remaining.
Delivery receipts	Every reminder logged as sent, uncertain or dashboard; unconfirmed deliveries are never auto-replayed and can be retried manually from Reminder history.
Export and backup	One-click JSON export of all records (never tokens or credentials).
Phone app (PWA)	Installable on Android over your own Wi-Fi: home-screen icon, standalone full-screen display, portrait layout.
Security built in	Localhost binding, host allow-list, CSRF token and origin checks, AES-256-GCM encrypted tokens, OAuth2 with PKCE. Section 10 details the model.
Time handling	Times stored in UTC, displayed in Asia/Karachi throughout - app, worksheet and reminders.
Automated tests	15 tests cover the store, the server and the worksheet sync. They run in temporary folders and never send real email or call real Google APIs.

SECTIONS 7 AND 9

Benefits - and what makes it unique

Benefits

- **Private by design.** Your records live in a file on your own computer. The server listens only to your own machine by default, every request passes host and CSRF checks, and your Google tokens are encrypted at rest. There is no vendor server to breach, leak or shut down.
- **Free, with no subscription.** No monthly fee, no per-user pricing, no feature tiers. The only account involved is your own Google account.
- **Nothing leaves except to your own Google.** The app sends data only to Gmail, Calendar, Drive and Sheets under your account, and only what each feature needs. No analytics, no advertising, no harvesting.
- **Offline-capable records.** Your schedule is a local database. With no internet at all you can still open the app, read every record and create new ones; Google features resume when the connection returns.
- **Invitations stop being manual work.** The inbox turns "someone emailed me a meeting" into a one-click decision, with the date already read out of the message.
- **Reminders you can audit.** Delivery receipts mean a reminder is never a matter of faith. If one was not confirmed, you can see that - and retry it yourself.
- **A worksheet that maintains itself.** The Drive spreadsheet is always current with zero effort, ready to open, share or print.
- **Beginner-friendly.** A double-click starts it, one click connected Google, and EASY-SETUP.md assumes no technical background at any step.

Uniqueness - how it differs from cloud schedulers

Cloud scheduler	This app
Your data on the vendor's servers	Your data in a SQLite file on your own disk
Monthly subscription for full features	Free; runs on software you already have
Email parsing, where offered, books automatically	Approval-first inbox: nothing is scheduled without your click
Reports are an export you run by hand	A Drive worksheet that updates itself within seconds
A failed notification is invisible	Every reminder carries a delivery receipt, with manual retry
Stops working if the vendor changes or closes	Plain Node.js files you own - Appendix A is the whole thing

SECTIONS 8 AND 10

Limitations - and the security model

Limitations, stated plainly

- **The computer must be on and the app running** for Gmail scanning and email reminders. The Node process must be alive and the machine awake and online; the browser itself may be closed. Events saved to Google still get Google's own reminders independently.
- **One manual Google consent click is unavoidable.** Google requires the account owner to approve the connection in person. No version of this app - or any honest app - can automate that away.
- **All-day Google events are not auto-imported.** They carry no start time, so there is nothing to remind about. Create a timed event manually if you need a reminder for one.
- **Date parsing is English only.** chrono-node reads English date expressions. Invitations in other languages land in the inbox without a reliable suggestion and are scheduled by hand.
- **Reminders can be late after downtime.** If the app or computer was off, the reminder is sent when it next runs, labelled with the true minutes remaining - and never after the event has started.
- **No WhatsApp control in version 2.** Removed completely at the owner's request. The app is operated from its web page and phone PWA only.
- **Single user, single machine.** It is built for one person's schedule on their own computer (plus their phone on the same Wi-Fi), not for a shared team calendar.

The security model

Protection	How it works
Local-only by default	The server binds to 127.0.0.1. Nothing on the network - let alone the internet - can reach it.
LAN mode is fenced	With ASIF_BIND=lan (phone access), the server accepts only loopback and the machine's own LAN addresses. Never port-forward it or expose it publicly.
Host allow-list	Every request's Host header is checked against the allowed list; anything else is rejected, which defeats DNS-rebinding tricks against local apps.
CSRF and origin checks	Non-GET requests must carry the matching Origin and the app's CSRF token, so a malicious web page cannot drive your scheduler from another tab.
Encrypted tokens	Google tokens and the OAuth client secret are sealed with AES-256-GCM; the key file has owner-only permissions.
OAuth done properly	Desktop-client flow with PKCE (S256) and a loopback-only redirect.
Private data folder	Records, database and key sit in one per-user folder (Section 11). Deleting it removes records and connections entirely; backing it up backs up everything.

SECTIONS 11 AND 12

Deployment and operating systems

Installing on another computer

1. Copy the app folder (or its zip) to the new computer - USB stick, email or cloud drive all work.
2. Install Node.js 24 LTS, extract the folder to a permanent location, and run the start script.
3. Either reconnect Google with the same one sign-in click, or copy the data folder across to carry your records and the encrypted connection with you.

Where your data lives

System	Data folder
Windows	`%LOCALAPPDATA%\AsifScheduleAssistant`
macOS / Linux	`~/AsifScheduleAssistant`
Override	Set the `ASIF_DATA_DIR` environment variable to any folder you prefer.

Backup guidance: copy that one folder somewhere safe, regularly. It holds the database, the settings and the encryption key together, so a restored copy restores everything. Keep backups private - the folder contains your schedule and your Google connection. The Settings page Export button is the lighter option: a JSON copy of the records alone.

Operating systems

System	How the app runs there
Windows 10 / 11	Full support. Double-click Start-Windows.cmd, or Start-Phone-Access.cmd for phone mode.
macOS	Full support. Run `bash start.sh`, then open the same local address in the browser.
Linux	Full support, same as macOS.
ChromeOS	Works through the Linux environment (Crostini) with Node.js 24 installed.
Android	PWA over your own Wi-Fi in LAN mode: open the printed address, then Add to Home screen.
iOS	The web page works in Safari over the LAN address; home-screen install behaviour varies by iOS version.

SECTION 13

The project files

Everything the app is made of fits in one folder. This is the map of that folder - the same files printed in full in Appendix A.

File	Lines	Role
`server.mjs`	274	HTTP server, API, Google OAuth, Gmail / Calendar / Drive / Sheets sync, reminder delivery
`core.mjs`	80	SQLite store, event validation, AES-256-GCM secret encryption, reminder scheduling
`sheets.mjs`	154	Google Sheets worksheet sync (Schedule + Dashboard tabs)
`public/index.html`	34	Page structure for all six views and the event form
`public/app.js`	123	Frontend logic
`public/style.css`	-	All styling, including the phone layout (not reprinted - it is presentation only)
`public/manifest.webmanifest`	1	PWA manifest (single-line JSON, as shipped)
`package.json`	17	Project definition: Node >= 24; dependencies googleapis and chrono-node
`Start-Windows.cmd` / `Start-Phone-Access.cmd` / `start.sh`	26 / 30 / 6	Start scripts for Windows, Windows phone mode, and macOS / Linux
`test/` (3 files)	-	15 automated tests: `npm test`

Dependencies: exactly two

The app installs only two libraries. `googleapis` is Google's own client library, used for Gmail, Calendar, Drive and Sheets. `chrono-node` reads English date expressions out of invitation text. Everything else - the HTTP server, the SQLite access, the encryption - comes from Node.js itself, which keeps the install small and the supply chain short.

The tests

Fifteen automated tests in three files (`core.test.mjs`, `server.test.mjs`, `sheets.test.mjs`) cover event validation and storage, the API and its security checks, and the worksheet layout. They run with `npm test`, use temporary folders, and never send a real email or call a real Google API - so they are safe to run at any time, and all fifteen pass on the delivered build.

> **Reading the code.** Appendix A prints the files in the order of this table, server first. A reader who works through server.mjs, then core.mjs, then sheets.mjs has read the entire backend; index.html and app.js are the entire frontend.

Appendix A - server.mjs (continued, 1 of 6) - lines 1-53 of 274

```
import http from 'node:http';
import fs from 'node:fs';
import os from 'node:os';
import path from 'node:path';
import {fileURLToPath} from 'node:url';
import {randomBytes,createHash} from 'node:crypto';
import {google} from 'googleapis';
import {Store,deliverDue,DEFAULTS} from './core.mjs';
import {SHEET_TITLE,SCHEDULE_TAB,DASHBOARD_TAB,HEADERS,buildRows,dashboardRows,layoutRequests,sheetUrl} from './sheets.mjs';

const root=path.dirname(fileURLToPath(import.meta.url));
const port=Number(process.env.PORT||4318);
// ASIF_BIND=lan lets the app be reached from your phone on the same Wi-Fi.
// Default stays localhost-only.
const bindLan=process.env.ASIF_BIND==='lan';
const listenAddr=bindLan?'0.0.0.0':'127.0.0.1';
const base=`http://127.0.0.1:${port}`;
function lanIPv4(){
  const out=[];
  for(const nets of Object.values(os.networkInterfaces()||{}))for(const n of nets||[]){
    if(n.family==='IPv4'&&!n.internal)out.push(n.address);
  }
  return out;
}
function allowedHosts(){
  return new Set([`127.0.0.1:${port}`,`localhost:${port}`,...lanIPv4().map(ip=>`${ip}:${port}`)]);
}
const data=process.env.ASIF_DATA_DIR||path.join(process.env.LOCALAPPDATA||process.env.HOME||root,'AsifScheduleAssistant');
const store=new Store(data);
const csrf=randomBytes(32).toString('hex');
let syncing=false,ticking=false;
let lastSync=null,syncError=null,sheetError=null,sheetSyncing=false,sheetTimer=null;
const flows=new Map();
const scopes=['https://www.googleapis.com/auth/gmail.readonly','https://www.googleapis.com/auth/gmail.send','https://www.googleapis.com/auth/calendar.events','https://www.googleapis.com/auth/drive.file','https://www.googleapis.com/auth/spreadsheets'];

function auth(persist=true){
  const cfg=store.secret('google_client');if(!cfg)throw new Error('Upload your Google Desktop OAuth client JSON in Connections first.');
  // Google sign-in always uses the loopback address, even when the app is
  // reached from a phone on the LAN (Google only allows loopback redirects
  // for Desktop clients). Do this step from the laptop browser.
  const a=new google.auth.OAuth2(cfg.client_id,cfg.client_secret,`http://127.0.0.1:${port}/oauth/google/callback`);
  const tokens=store.secret('google_tokens');if(tokens)a.setCredentials(tokens);
  if(persist)a.on('tokens',t=>store.secret('google_tokens',{...store.secret('google_tokens'),...t}));return a;
}
function googleReady(){return !!store.secret('google_tokens');}
function textBody(payload){
  const found=[];const visit=p=>{if(p.mimeType==='text/plain'&&p.body?.data)found.push(Buffer.from(p.body.data,'base64url').toString());for(const x of p.parts||[])visit(x);};visit(payload);return found.join('\n').slice(0,6000);
}
async function syncGoogle(){
  if(syncing||!googleReady())return;syncing=true;syncError=null;
  try{
    const a=auth(),calendar=google.calendar({version:'v3',auth:a});let pageToken;
    for(let page=0;page<10;page++){
```

Appendix A - server.mjs (continued, 2 of 6) - lines 54-107 of 274

```
      const r=await calendar.events.list({calendarId:'primary',timeMin:new Date(Date.now()-86400000).toISOString(),timeMax:new Date(Date.now()+90*86400000).toISOString(),singleEvents:true,showDeleted:true,maxResults:250,pageToken});
      for(const e of r.data.items||[])store.importGoogle(e);pageToken=r.data.nextPageToken;if(!pageToken)break;
    }
    const gmail=google.gmail({version:'v1',auth:a});
    const r=await gmail.users.messages.list({userId:'me',maxResults:40,q:'newer_than:14d -in:sent -in:trash (meeting OR appointment OR schedule OR webinar OR invitation OR "zoom.us" OR "meet.google.com")'});
    for(const m of r.data.messages||[]){
      if(store.db.prepare("SELECT id FROM inbox WHERE source='gmail' AND source_id=?").get(m.id))continue;
      const full=await gmail.users.messages.get({userId:'me',id:m.id,format:'full'});
      const h=full.data.payload?.headers||[];const title=h.find(x=>x.name.toLowerCase()==='subject')?.value||'Email invitation';
      const body=textBody(full.data.payload||{})||full.data.snippet||'';
      store.suggestion({source:'gmail',source_id:m.id,title,body:title+'\n'+body,received:new Date(Number(full.data.internalDate)||Date.now())});
    }
    lastSync=new Date().toISOString();store.set('last_sync',lastSync);scheduleSheetSync();
  }catch(e){syncError='Google sync failed. Check your sign-in, API access and internet connection.';}
  finally{syncing=false;}
}
// ---- Google Sheets worksheet: every schedule entry is kept as a row ----
async function ensureSheet(){
  let id = store.get('sheet_id');
  const a = auth(), sheets = google.sheets({version:'v4', auth:a});
  if(id){
    try{ await sheets.spreadsheets.get({spreadsheetId:id, fields:'spreadsheetId'}); return id; }
    catch{ id = null; }
  }
  const drive = google.drive({version:'v3', auth:a});
  const found = await drive.files.list({
    q:`name='${SHEET_TITLE}' and mimeType='application/vnd.google-apps.spreadsheet' and trashed=false`,
    fields:'files(id)', pageSize:1,
  });
  if(found.data.files && found.data.files.length){ id = found.data.files[0].id; }
  else{
    const created = await sheets.spreadsheets.create({requestBody:{
      properties:{title:SHEET_TITLE, locale:'en_PK', timeZone:'Asia/Karachi'},
      sheets:[{properties:{title:SCHEDULE_TAB}},{properties:{title:DASHBOARD_TAB}}],
    }});
    id = created.data.spreadsheetId;
  }
  store.set('sheet_id', id);
  await setupSheetLayout(id);
  return id;
}
async function setupSheetLayout(id){
  const a = auth(), sheets = google.sheets({version:'v4', auth:a});
  const meta = await sheets.spreadsheets.get({spreadsheetId:id, fields:'sheets.properties'});
  const tabId = (title)=>{
    const s = (meta.data.sheets||[]).find(x=>x.properties.title===title);
    return s ? s.properties.sheetId : null;
  };
  let scheduleId = tabId(SCHEDULE_TAB), dashboardId = tabId(DASHBOARD_TAB);
  const add = [];
  if(scheduleId===null) add.push({addSheet:{properties:{title:SCHEDULE_TAB}}});
  if(dashboardId===null) add.push({addSheet:{properties:{title:DASHBOARD_TAB}}});
  if(add.length){
    const r = await sheets.spreadsheets.batchUpdate({spreadsheetId:id, requestBody:{requests:add}});
```

Appendix A - server.mjs (continued, 3 of 6) - lines 108-163 of 274

```
    for(const x of r.data.replies||[]){
      if(x.addSheet.properties.title===SCHEDULE_TAB) scheduleId = x.addSheet.properties.sheetId;
      if(x.addSheet.properties.title===DASHBOARD_TAB) dashboardId = x.addSheet.properties.sheetId;
    }
  }
  await sheets.spreadsheets.values.update({
    spreadsheetId:id, range:`${SCHEDULE_TAB}!A1`, valueInputOption:'RAW',
    requestBody:{values:[HEADERS]},
  });
  await sheets.spreadsheets.values.update({
    spreadsheetId:id, range:`${DASHBOARD_TAB}!A1`, valueInputOption:'USER_ENTERED',
    requestBody:{values:dashboardRows()},
  });
  await sheets.spreadsheets.batchUpdate({
    spreadsheetId:id, requestBody:{requests:layoutRequests(scheduleId, dashboardId)},
  });
}
async function syncSheet(){
  if(sheetSyncing || !googleReady()) return;
  sheetSyncing = true; sheetError = null;
  try{
    const id = await ensureSheet();
    const sheets = google.sheets({version:'v4', auth:auth()});
    const rows = buildRows(store.events());
    await sheets.spreadsheets.values.clear({spreadsheetId:id, range:`${SCHEDULE_TAB}!A2:L`});
    if(rows.length){
      await sheets.spreadsheets.values.update({
        spreadsheetId:id, range:`${SCHEDULE_TAB}!A2:L${rows.length+1}`,
        valueInputOption:'USER_ENTERED', requestBody:{values:rows},
      });
    }
    const stamp = new Intl.DateTimeFormat('en-PK',{dateStyle:'medium',timeStyle:'short',timeZone:'Asia/Karachi'}).format(new Date());
    await sheets.spreadsheets.values.update({
      spreadsheetId:id, range:`${DASHBOARD_TAB}!B10`, valueInputOption:'RAW',
      requestBody:{values:[[stamp + ' (Pakistan time)']]},
    });
    store.set('sheet_last', new Date().toISOString());
  }catch(e){
    sheetError = 'Worksheet sync failed. Check your Google sign-in and that Drive/Sheets APIs are enabled.';
  }finally{ sheetSyncing = false; }
}
// Debounced: many quick edits trigger a single sheet update a few seconds later.
function scheduleSheetSync(){
  if(sheetTimer || !googleReady()) return;
  sheetTimer = setTimeout(()=>{ sheetTimer = null; void syncSheet(); }, 4000);
}
async function sendEmail(e,message){
  const to=store.settings().email;if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(to)||/[\r\n]/.test(to))throw new Error('Invalid recipient');
  const raw=Buffer.from(`To: ${to}\r\nSubject: =?UTF-8?B?${Buffer.from('30-minute reminder: '+e.title).toString('base64')}?=\r\nMIME-Version: 1.0\r\nContent-Type: text/plain; charset=UTF-8\r\nContent-Transfer-Encoding: base64\r\n\r\n${Buffer.from(message).toString('base64')}\r\n`).toString('base64url');
  await google.gmail({version:'v1',auth:auth()}).users.messages.send({userId:'me',requestBody:{raw}});
}
async function tick(){
  if(ticking)return;ticking=true;
  try{await deliverDue(store,{email:googleReady()?sendEmail:null});}
  finally{ticking=false;}
}
```

Appendix A - server.mjs (continued, 4 of 6) - lines 164-205 of 274

```
function respond(res,status,value){res.writeHead(status,{'Content-Type':'application/json','Cache-Control':'no-store'});res.end(JSON.stringify(value));}
async function body(req){let b='';for await(const x of req){b+=x;if(b.length>150000)throw new Error('Request too large.');}return b?JSON.parse(b):{};}
function security(req,res){
  res.setHeader('X-Content-Type-Options','nosniff');res.setHeader('Referrer-Policy','no-referrer');res.setHeader('X-Frame-Options','DENY');
  const host=req.headers.host;
  if(!allowedHosts().has(host)){respond(res,403,{error:'Use the app address shown by the app on this computer.'});return false;}
  const reqBase=`http://${host}`;
  if(!['GET','HEAD'].includes(req.method)&&(req.headers.origin!==reqBase||req.headers['x-asif-csrf']!==csrf)){respond(res,403,{error:'This request must come from the app.'});return false;}
  return true;
}
const STATIC_FILES={
  '/':['index.html','text/html'],
  '/app.js':['app.js','text/javascript'],
  '/style.css':['style.css','text/css'],
  '/manifest.webmanifest':['manifest.webmanifest','application/manifest+json'],
  '/icon-192.png':['icon-192.png','image/png'],
  '/icon-512.png':['icon-512.png','image/png'],
};
const server=http.createServer(async(req,res)=>{
  if(!security(req,res))return;
  const url=new URL(req.url,base);
  try{
    if(req.method==='GET'&&url.pathname==='/api/state'){const sheetId=store.get('sheet_id');return respond(res,200,{csrf,settings:store.settings(),events:store.events(),inbox:store.inbox(),logs:store.logs(),connections:{google:{configured:!!store.secret('google_client'),connected:googleReady(),email:store.get('google_email')}},sheet:sheetId?{id:sheetId,url:sheetUrl(sheetId),last:store.get('sheet_last'),error:sheetError,busy:sheetSyncing}:null,sync:{busy:syncing,last:lastSync||store.get('last_sync'),error:syncError},mode:'desktop'});}
    if(req.method==='GET'&&url.pathname==='/oauth/google/callback'){
      const state=url.searchParams.get('state'),flow=flows.get(state);flows.delete(state);
      if(!flow||flow.expires<Date.now())throw new Error('Google sign-in expired. Start again from Connections.');
      if(url.searchParams.has('error'))throw new Error('Google sign-in was cancelled.');
      const a=auth(false);const {tokens}=await a.getToken({code:url.searchParams.get('code'),codeVerifier:flow.verifier,redirect_uri:`http://127.0.0.1:${port}/oauth/google/callback`});
      a.setCredentials(tokens);const profile=await google.gmail({version:'v1',auth:a}).users.getProfile({userId:'me'});
      if(profile.data.emailAddress.toLowerCase()!==store.settings().email.toLowerCase())throw new Error('Choose '+store.settings().email+' when signing in, or update your email in Settings first.');
      store.secret('google_tokens',tokens);store.set('google_email',profile.data.emailAddress);void syncGoogle();res.writeHead(302,{Location:'/#connections'});return res.end();
    }
    if(req.method==='POST'&&url.pathname==='/api/google/config'){
      const b=await body(req);const cfg=b.installed;
      if(!cfg?.client_id||!cfg?.client_secret)throw new Error('Upload a Google OAuth JSON for a Desktop app (installed), not a service account.');
      store.secret('google_client',{client_id:cfg.client_id,client_secret:cfg.client_secret});return respond(res,200,{ok:true});
    }
    if(req.method==='POST'&&url.pathname==='/api/google/connect'){
      const a=auth(),state=randomBytes(24).toString('base64url'),verifier=randomBytes(48).toString('base64url');flows.set(state,{verifier,expires:Date.now()+10*60000});
      return respond(res,200,{url:a.generateAuthUrl({access_type:'offline',prompt:'consent',scope:scopes,state,login_hint:store.settings().email,code_challenge:createHash('sha256').update(verifier).digest('base64url'),code_challenge_method:'S256'})});
    }
    if(req.method==='POST'&&url.pathname==='/api/google/disconnect'){const tokens=store.secret('google_tokens');if(tokens?.access_token)await auth().revokeToken(tokens.access_token).catch(()=>{});store.secret('google_tokens',null);store.set('google_email',null);return respond(res,200,{ok:true});}
```

Appendix A - server.mjs (continued, 5 of 6) - lines 206-246 of 274

```
206     if(req.method==='POST'&&url.pathname==='/api/sync'){void syncGoogle();return respond(res,202,{ok:t
    rue});}
207     if(req.method==='POST'&&url.pathname==='/api/sheet/sync'){void syncSheet();return respond(res,202,
    {ok:true});}
208     if(req.method==='POST'&&url.pathname==='/api/settings'){
209       const b=await body(req),old=store.settings();const email=String(b.email||old.email).trim();
210       if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)||/[\r\n]/.test(email))throw new Error('Enter a vali
    d email.');
211       if(googleReady()&&email!==old.email)throw new Error('Disconnect Google before changing your emai
    l.');
212       store.set('preferences',{...old,email,emailReminders:b.emailReminders!==false,autoSync:b.autoSyn
    c!==false});return respond(res,200,{ok:true});
213     }
214     if(req.method==='POST'&&url.pathname==='/api/events'){const b=await body(req);const e=store.create
    (b);if(b.inboxId)store.review(b.inboxId,'accepted');scheduleSheetSync();return respond(res,201,e);}
215     const m=url.pathname.match(/^\/api\/events\/([^/]+)(?:\/(.*))?$/);
216     if(m&&req.method==='POST'){
217       const id=decodeURIComponent(m[1]),e=store.event(id);if(!e)return respond(res,404,{error:'Meeting
    not found.'});
218       const b=await body(req);
219       if(!m[2]){
220         if(e.google_id){
221           if(!googleReady())throw new Error('Connect Google before editing this linked event.');
222           const {validateEvent}=await import('./core.mjs');const value=validateEvent(b);
223           await google.calendar({version:'v3',auth:auth()}).events.patch({calendarId:'primary',eventI
    d:e.google_id,requestBody:{summary:value.title,description:value.notes,location:value.location,start:
    {dateTime:value.start,timeZone:'Asia/Karachi'},end:{dateTime:value.end,timeZone:'Asia/Karachi'}},sendU
    pdates:'none'});
224         }
225         const updated=store.edit(id,b);scheduleSheetSync();return respond(res,200,updated);
226       }
227       if(m[2]==='status'){
228         if(!['scheduled','completed','cancelled'].includes(b.status))throw new Error('Enter a valid ev
    ent status.');
229         if(b.status==='cancelled'&&e.google_id){if(!googleReady())throw new Error('Connect Google to c
    ancel this linked event and stop its Google reminders.');await google.calendar({version:'v3',auth:auth
    ()}).events.delete({calendarId:'primary',eventId:e.google_id,sendUpdates:'none'});}
230         store.mark(id,b.status);scheduleSheetSync();return respond(res,200,{ok:true});
231       }
232       if(m[2]==='google'){
233         if(!googleReady())throw new Error('Connect Google first.');const calendar=google.calendar({ver
    sion:'v3',auth:auth()});
234         const payload={summary:e.title,description:e.notes,location:e.location,start:{dateTime:e.star
    t,timeZone:'Asia/Karachi'},end:{dateTime:e.end,timeZone:'Asia/Karachi'},reminders:{useDefault:false,ov
    errides:[{method:'popup',minutes:30},{method:'email',minutes:30}]}};
235         if(e.google_id)await calendar.events.patch({calendarId:'primary',eventId:e.google_id,requestBo
    dy:payload,sendUpdates:'none'});
236         else{
237           const googleId='asif'+id.replace(/-/g,'');
238           try{await calendar.events.insert({calendarId:'primary',requestBody:{...payload,id:googleId},
    sendUpdates:'none'});}
239           catch(error){if(error.code!==409)throw error;await calendar.events.get({calendarId:'primar
    y',eventId:googleId});}
240           store.google(id,googleId);
241         }
242         return respond(res,200,{ok:true});
243       }
244     }
245     if(req.method==='POST'&&url.pathname.startsWith('/api/reminders/')){
246       const id=url.pathname.split('/')[3];const log=store.db.prepare('SELECT * FROM deliveries WHERE i
    d=?').get(id);if(!log||log.status!=='uncertain')throw new Error('Enter an unconfirmed reminder to retr
    y.');
```

Appendix A - server.mjs (continued, 6 of 6) - lines 247-274 of 274

```
      const e=store.event(log.event_id);if(!e||e.status!=='scheduled'||Date.parse(e.start)<=Date.now()||e.start!==log.event_start)throw new Error('Meeting changed or has already started; this reminder cannot be retried.');
      const send=log.channel==='email'&&googleReady()?sendEmail:null;if(!send)throw new Error('Connect Google before retrying.');
      store.delivery(e,log.channel,'sending');
      try{const {reminderMessage}=await import('./core.mjs');await send(e,reminderMessage(e));store.delivery(e,log.channel,'sent','Provider confirmed acceptance after manual retry.');}
      catch(error){store.delivery(e,log.channel,'uncertain','Delivery was not confirmed after manual retry.');}
      return respond(res,200,{ok:true});
    }
    if(req.method==='POST'&&url.pathname==='/api/inbox/dismiss'){const b=await body(req);store.review(b.id,'dismissed');return respond(res,200,{ok:true});}
    if(req.method==='POST'&&url.pathname==='/api/import-text'){const b=await body(req);if(!String(b.text||'').trim())throw new Error('Paste a meeting invitation.');store.suggestion({source:['email','text'].includes(b.source)?b.source:'text',source_id:randomBytes(12).toString('hex'),title:String(b.title||'Invitation to review'),body:String(b.text),received:new Date()});return respond(res,201,{ok:true});}
    if(req.method==='GET'&&url.pathname==='/api/export'){res.writeHead(200,{'Content-Type':'application/json','Content-Disposition':'attachment; filename="asif-schedule-backup.json"','Cache-Control':'no-store'});return res.end(JSON.stringify({exported:new Date().toISOString(),events:store.events(),inbox:store.inbox(),logs:store.logs()},null,2));}
    if(req.method==='GET'&&url.pathname==='/setup'){res.writeHead(200,{'Content-Type':'text/plain; charset=utf-8'});return fs.createReadStream(path.join(root,'SETUP.md')).pipe(res);}
    if(req.method==='GET'&&STATIC_FILES[url.pathname]){
      const [file,type]=STATIC_FILES[url.pathname];
      res.writeHead(200,{'Content-Type':type,'Cache-Control':'no-store'});return fs.createReadStream(path.join(root,'public',file)).pipe(res);
    }
    respond(res,404,{error:'Not found.'});
  }catch(e){respond(res,400,{error:e.message?.startsWith('Choose ')||/^(Enter|Upload|Connect|Google sign-in|The scanned|Meeting|Disconnect|An event|Request|This|Paste)/.test(e.message||'')?e.message:'The action could not be completed. Check your connection and try again.'});}
});
server.listen(port,listenAddr,()=>{
  const lan=bindLan?lanIPv4():[];
  console.log(`\nAsif Schedule Assistant\nOpen ${base}\n`+(lan.length?`On your phone (same Wi-Fi), open: ${lan.map(ip=>`http://${ip}:${port}`).join('  or  ')}\n`:'')+`Keep this process running for Gmail reminders.\n`);
  if(googleReady())void syncGoogle();
});
const reminderTimer=setInterval(()=>void tick().catch(()=>{}),15000);
const syncTimer=setInterval(()=>{if(store.settings().autoSync)void syncGoogle();},5*60000);
void tick().catch(()=>{});
async function shutdown(){clearInterval(reminderTimer);clearInterval(syncTimer);if(sheetTimer)clearTimeout(sheetTimer);server.close();store.close();process.exit(0);}
process.on('SIGINT',()=>void shutdown());process.on('SIGTERM',()=>void shutdown());
```

Appendix A - core.mjs (continued, 1 of 2) - lines 1-42 of 80

```
import {DatabaseSync} from 'node:sqlite';
import {randomUUID, randomBytes, createCipheriv, createDecipheriv} from 'node:crypto';
import fs from 'node:fs';
import path from 'node:path';
import * as chrono from 'chrono-node';

export const DEFAULTS={name:'Prof. Dr. Asif Ahmad',email:'asifahmad1@gmail.com',timezone:'Asia/Karachi',emailReminders:true,autoSync:true};
export function validateEvent(input){
  const title=String(input.title??'').trim();
  if(!title||title.length>180) throw new Error('Enter a title of 1–180 characters.');
  const start=new Date(input.start); const end=new Date(input.end??start.getTime()+3600000);
  if(!Number.isFinite(start.getTime())||!Number.isFinite(end.getTime())||end<=start) throw new Error('Choose a valid start and end time.');
  if(end-start>7*86400000) throw new Error('An event cannot be longer than seven days.');
  const kind=['meeting','task','appointment'].includes(input.kind)?input.kind:'meeting';
  return {title,start:start.toISOString(),end:end.toISOString(),kind,location:String(input.location??'').slice(0,500),notes:String(input.notes??'').slice(0,5000)};
}
export function parseSuggestion(text,reference=new Date()){
  const parsed=chrono.parse(String(text).slice(0,6000),{instant:new Date(reference),timezone:300},{forwardDate:true,timezones:{PKT:300}})[0];
  if(!parsed) return {start:null,end:null,confidence:'Review date and time'};
  const start=parsed.start.date(); const end=parsed.end?.date()??new Date(start.getTime()+3600000);
  return {start:start.toISOString(),end:end.toISOString(),confidence:parsed.start.isCertain('hour')?'Review suggested date and time':'Time missing — please choose a time'};
}
export function reminderMessage(e,now=Date.now()){
  const mins=Math.max(0,Math.ceil((Date.parse(e.start)-now)/60000));
  const date=new Intl.DateTimeFormat('en-PK',{dateStyle:'medium',timeStyle:'short',timeZone:'Asia/Karachi'}).format(new Date(e.start));
  return `Reminder: ${e.title}\nStarts in ${mins} minutes\n${date} (Pakistan time)${e.location?'\nLocation: '+e.location:''}${e.notes?'\n'+e.notes.slice(0,700):''}`;
}
export class Store{
  constructor(dir){
    fs.mkdirSync(dir,{recursive:true,mode:0o700}); this.dir=dir;
    this.db=new DatabaseSync(path.join(dir,'schedule.sqlite')); this.db.exec('PRAGMA journal_mode=WAL; PRAGMA busy_timeout=5000;');
    this.db.exec(`CREATE TABLE IF NOT EXISTS settings (key TEXT PRIMARY KEY,value TEXT NOT NULL);
      CREATE TABLE IF NOT EXISTS events (id TEXT PRIMARY KEY,title TEXT NOT NULL,start TEXT NOT NULL,end TEXT NOT NULL,kind TEXT NOT NULL,location TEXT NOT NULL DEFAULT '',notes TEXT NOT NULL DEFAULT '',status TEXT NOT NULL DEFAULT 'scheduled',source TEXT NOT NULL DEFAULT 'manual',source_id TEXT,google_id TEXT,created TEXT NOT NULL);
      CREATE UNIQUE INDEX IF NOT EXISTS source_unique ON events(source,source_id) WHERE source_id IS NOT NULL;
      CREATE TABLE IF NOT EXISTS inbox (id TEXT PRIMARY KEY,source TEXT NOT NULL,source_id TEXT NOT NULL,title TEXT NOT NULL,body TEXT NOT NULL,suggested_start TEXT,suggested_end TEXT,confidence TEXT,received TEXT NOT NULL,status TEXT NOT NULL DEFAULT 'review',UNIQUE(source,source_id));
      CREATE TABLE IF NOT EXISTS deliveries (id TEXT PRIMARY KEY,event_id TEXT NOT NULL,event_start TEXT NOT NULL,channel TEXT NOT NULL,status TEXT NOT NULL,detail TEXT,created TEXT NOT NULL,updated TEXT NOT NULL,UNIQUE(event_id,event_start,channel));`);
    this.db.prepare("UPDATE deliveries SET status='uncertain',detail='App restarted before provider confirmation; review before retrying.' WHERE status='sending'").run();
    const keyFile=path.join(dir,'local.key'); if(!fs.existsSync(keyFile))fs.writeFileSync(keyFile,randomBytes(32),{mode:0o600}); this.key=fs.readFileSync(keyFile);
  }
  get(key,fallback=null){const r=this.db.prepare('SELECT value FROM settings WHERE key=?').get(key); return r?JSON.parse(r.value):fallback;}
  set(key,value){this.db.prepare('INSERT INTO settings VALUES (?,?) ON CONFLICT(key) DO UPDATE SET value=excluded.value').run(key,JSON.stringify(value));}
  seal(value){const iv=randomBytes(12);const c=createCipheriv('aes-256-gcm',this.key,iv);const encrypted=Buffer.concat([c.update(JSON.stringify(value)),c.final()]);return Buffer.concat([iv,encrypted,c.getAuthTag()]).toString('base64');}
```

Appendix A - core.mjs (continued, 2 of 2) - lines 43-80 of 80

```
43   unseal(value){const b=Buffer.from(value,'base64');const d=createDecipheriv('aes-256-gcm',this.key,b.
subarray(0,12));d.setAuthTag(b.subarray(-16));return JSON.parse(Buffer.concat([d.update(b.subarray(12,
-16)),d.final()]).toString());}
44   secret(key,value){if(value!==undefined)this.set(key,value===null?null:this.seal(value)); const v=thi
s.get(key);return v?this.unseal(v):null;}
45   settings(){return {...DEFAULTS,...this.get('preferences',{})};}
46   events(){return this.db.prepare('SELECT * FROM events ORDER BY start DESC').all();}
47   event(id){return this.db.prepare('SELECT * FROM events WHERE id=?').get(id);}
48   create(input,meta={}){const e=validateEvent(input);const id=meta.id??randomUUID();this.db.prepare('I
NSERT INTO events(id,title,start,end,kind,location,notes,source,source_id,google_id,created) VALUES
(?,?,?,?,?,?,?,?,?,?,?)').run(id,e.title,e.start,e.end,e.kind,e.location,e.notes,meta.source??'manua
l',meta.source_id??null,meta.google_id??null,new Date().toISOString());return this.event(id);}
49   edit(id,input){if(!this.event(id))throw new Error('Meeting not found.');const e=validateEvent(inpu
t);this.db.prepare('UPDATE events SET title=?,start=?,end=?,kind=?,location=?,notes=? WHERE id=?').run
(e.title,e.start,e.end,e.kind,e.location,e.notes,id);return this.event(id);}
50   mark(id,status){if(!['scheduled','completed','cancelled'].includes(status))throw new Error('Invalid
status.');this.db.prepare('UPDATE events SET status=? WHERE id=?').run(status,id);}
51   google(id,googleId){this.db.prepare('UPDATE events SET google_id=? WHERE id=?').run(googleId,id);}
52   importGoogle(e){
53     if(e.status==='cancelled'){this.db.prepare("UPDATE events SET status='cancelled' WHERE google_id
=?").run(e.id);return;}
54     if(!e.start?.dateTime||!e.end?.dateTime)return; // All-day events need an explicit time before rem
inders.
55     const found=this.db.prepare('SELECT id,kind FROM events WHERE google_id=?').get(e.id);
56     const value={title:e.summary||'Untitled meeting',start:e.start.dateTime,end:e.end.dateTime,locatio
n:e.location,notes:e.description,kind:found?.kind||'meeting'};
57     if(found)this.edit(found.id,value);else this.create(value,{source:'google',source_id:e.id,google_i
d:e.id});
58   }
59   suggestion({source,source_id,title,body,received}){const p=parseSuggestion(body,received);this.db.pr
epare('INSERT OR IGNORE INTO inbox(id,source,source_id,title,body,suggested_start,suggested_end,confid
ence,received) VALUES (?,?,?,?,?,?,?,?,?)').run(randomUUID(),source,source_id,title.slice(0,180),body.
slice(0,6000),p.start,p.end,p.confidence,new Date(received).toISOString());}
60   inbox(){return this.db.prepare("SELECT * FROM inbox WHERE status='review' ORDER BY received DESC LIM
IT 100").all();}
61   review(id,status){this.db.prepare('UPDATE inbox SET status=? WHERE id=?').run(status,id);}
62   claim(e,channel,now=Date.now()){
63     const stamp=new Date(now).toISOString(); const r=this.db.prepare("INSERT OR IGNORE INTO deliveries
(id,event_id,event_start,channel,status,created,updated) VALUES (?,?,?,?,'sending',?,?)").run(randomUU
ID(),e.id,e.start,channel,stamp,stamp);return Number(r.changes)>0;
64   }
65   delivery(e,channel,status,detail=''){this.db.prepare('UPDATE deliveries SET status=?,detail=?,update
d=? WHERE event_id=? AND event_start=? AND channel=?').run(status,String(detail).slice(0,500),new Date
().toISOString(),e.id,e.start,channel);}
66   logs(){return this.db.prepare('SELECT d.*,e.title FROM deliveries d LEFT JOIN events e ON e.id=d.eve
nt_id ORDER BY d.created DESC LIMIT 100').all();}
67   due(now=Date.now()){return this.db.prepare("SELECT * FROM events WHERE status='scheduled' AND start
>? AND start<=?").all(new Date(now).toISOString(),new Date(now+30*60000).toISOString());}
68   close(){this.db.close();}
69 }
70 export async function deliverDue(store,providers,now=Date.now()){
71   const settings=store.settings();
72   for(const e of store.due(now)){
73     for(const channel of ['dashboard','email']){
74       if(channel==='email'&&(!settings.emailReminders||!providers.email))continue;
75       if(!store.claim(e,channel,now))continue;
76       try{if(channel!=='dashboard')await providers[channel](e,reminderMessage(e,now));store.delivery
(e,channel,'sent',channel==='dashboard'?'Reminder recorded in dashboard.':'Provider confirmed acceptan
ce.');}
77       catch(error){store.delivery(e,channel,'uncertain','Delivery was not confirmed. Review connectio
n; no automatic replay.');}
78     }
79   }
80 }
```

Appendix A - sheets.mjs (continued, 1 of 3) - lines 1-63 of 154

```
// Google Sheets worksheet helpers for Asif Schedule Assistant.
// Pure functions only (no network): easy to unit-test. server.mjs handles
// the live Sheets/Drive API calls.
export const SHEET_TITLE = 'Asif Schedule Assistant';
export const SCHEDULE_TAB = 'Schedule';
export const DASHBOARD_TAB = 'Dashboard';

export const HEADERS = [
  'Title', 'Type', 'Date', 'Start', 'End', 'Duration (min)',
  'Location', 'Status', 'Reminder at', 'Notes', 'Source', 'Updated',
];

function pkDate(iso) {
  return new Intl.DateTimeFormat('en-CA', {
    timeZone: 'Asia/Karachi', year: 'numeric', month: '2-digit', day: '2-digit',
  }).format(new Date(iso));
}

function pkTime(iso) {
  return new Intl.DateTimeFormat('en-GB', {
    timeZone: 'Asia/Karachi', hour: '2-digit', minute: '2-digit', hourCycle: 'h23',
  }).format(new Date(iso));
}

/** One row per event, oldest first. Dates/times are Pakistan time. */
export function buildRows(events) {
  return [...events]
    .sort((a, b) => (a.start < b.start ? -1 : 1))
    .map((e) => {
      const dur = Math.round((Date.parse(e.end) - Date.parse(e.start)) / 60000);
      const updated = e.created || new Date().toISOString();
      return [
        e.title,
        e.kind,
        pkDate(e.start),
        pkTime(e.start),
        pkTime(e.end),
        dur,
        e.location || '',
        e.status,
        pkTime(Date.parse(e.start) - 30 * 60000),
        (e.notes || '').slice(0, 500),
        e.source || 'manual',
        `${pkDate(updated)} ${pkTime(updated)}`,
      ];
    });
}

/** Dashboard tab content: labels plus live formulas. */
export function dashboardRows() {
  return [
    ['Asif Schedule Assistant — created by Dr. Asif Ahmad'],
    [],
    ['Metric', 'Value'],
    ['Total entries', `=COUNTA(${SCHEDULE_TAB}!A:A)-1`],
    ['Upcoming (scheduled)', `=COUNTIFS(${SCHEDULE_TAB}!H:H,"scheduled",${SCHEDULE_TAB}!C:C,">="&TODAY
())`],
    ['Due today', `=COUNTIFS(${SCHEDULE_TAB}!H:H,"scheduled",${SCHEDULE_TAB}!C:C,"="&TODAY())`],
    ['Completed', `=COUNTIF(${SCHEDULE_TAB}!H:H,"completed")`],
    ['Cancelled', `=COUNTIF(${SCHEDULE_TAB}!H:H,"cancelled")`],
    [],
    ['Last synced', ''],
  ];
}
```

```
64
65  /** batchUpdate requests: tabs, header style, filters, status colours, widths. */
66  export function layoutRequests(scheduleSheetId, dashboardSheetId) {
67    const headerBg = { red: 0.145, green: 0.22, blue: 0.26 };
68    const white = { red: 1, green: 1, blue: 1 };
69    return [
70      // Header row: dark background, white bold text
71      {
72        repeatCell: {
73          range: { sheetId: scheduleSheetId, startRowIndex: 0, endRowIndex: 1 },
74          cell: {
75            userEnteredFormat: {
76              backgroundColor: headerBg,
77              textFormat: { foregroundColor: white, bold: true },
78            },
79          },
80          fields: 'userEnteredFormat(backgroundColor,textFormat)',
81        },
82      },
83      // Freeze header + add filter
84      {
85        updateSheetProperties: {
86          properties: { sheetId: scheduleSheetId, gridProperties: { frozenRowCount: 1 } },
87          fields: 'gridProperties.frozenRowCount',
88        },
89      },
90      {
91        setBasicFilter: {
92          filter: {
93            range: {
94              sheetId: scheduleSheetId, startRowIndex: 0, endRowIndex: 1,
95              startColumnIndex: 0, endColumnIndex: HEADERS.length,
96            },
97          },
98        },
99      },
100     // Status colours: scheduled = green, cancelled = red, completed = blue
101     ...[
102       ['scheduled', { red: 0.85, green: 0.95, blue: 0.87 }],
103       ['cancelled', { red: 0.98, green: 0.88, blue: 0.87 }],
104       ['completed', { red: 0.87, green: 0.91, blue: 0.97 }],
105     ].map(([status, bg]) => ({
106       addConditionalFormatRule: {
107         rule: {
108           ranges: [{ sheetId: scheduleSheetId, startColumnIndex: 7, endColumnIndex: 8 }],
109           booleanRule: {
110             condition: { type: 'TEXT_EQ', values: [{ userEnteredValue: status }] },
111             format: { backgroundColor: bg },
112           },
113         },
114         index: 0,
115       },
116     })),
117     // Column widths
118     {
119       updateDimensionProperties: {
120         range: { sheetId: scheduleSheetId, dimension: 'COLUMNS', startIndex: 0, endIndex: 1 },
121         properties: { pixelSize: 260 },
122         fields: 'pixelSize',
123       },
124     },
125     {
126       updateDimensionProperties: {
127         range: { sheetId: scheduleSheetId, dimension: 'COLUMNS', startIndex: 9, endIndex: 10 },
```

Appendix A - sheets.mjs (continued, 3 of 3) - lines 128-154 of 154

```
        properties: { pixelSize: 320 },
        fields: 'pixelSize',
      },
    },
    // Dashboard title style
    {
      repeatCell: {
        range: { sheetId: dashboardSheetId, startRowIndex: 0, endRowIndex: 1 },
        cell: {
          userEnteredFormat: { textFormat: { bold: true, fontSize: 14 } },
        },
        fields: 'userEnteredFormat(textFormat)',
      },
    },
    {
      updateDimensionProperties: {
        range: { sheetId: dashboardSheetId, dimension: 'COLUMNS', startIndex: 0, endIndex: 2 },
        properties: { pixelSize: 220 },
        fields: 'pixelSize',
      },
    },
  ];
}

export function sheetUrl(id) {
  return `https://docs.google.com/spreadsheets/d/${id}`;
}
```

Appendix A - public/index.html (continued, 1 of 3) - lines 1-17 of 34

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><meta name="description" content="Your private meeting calendar, invitation inbox, and 30-minute reminder assistant."><title>Asif Schedule Assistant</title><link rel="stylesheet" href="/style.css"><link rel="manifest" href="/manifest.webmanifest"><meta name="theme-color" content="#253842"><meta name="mobile-web-app-capable" content="yes"><meta name="apple-mobile-web-app-capable" content="yes"><meta name="apple-mobile-web-app-title" content="Schedule"><link rel="apple-touch-icon" href="/icon-192.png"><link rel="icon" type="image/png" href="/icon-192.png"></head>
<body>
<aside class="sidebar"><a class="brand" href="#overview"><span class="brand-icon">a<span>·</span></span><span>asif<span class="brand-sub">schedule assistant</span></span></a>
<p class="nav-label">YOUR WORKSPACE</p><nav>
<a href="#overview" data-page="overview"><span>◫</span> Overview</a><a href="#agenda" data-page="agenda"><span>▦</span> My schedule</a><a href="#inbox" data-page="inbox"><span>▤</span> Invitation inbox <b id="inbox-count">0</b></a><a href="#connections" data-page="connections"><span>⛓</span> Connections</a><a href="#history" data-page="history"><span>◷</span> Reminder history</a><a href="#settings" data-page="settings"><span>⚙</span> Settings</a></nav>
<div class="sidebar-note"><span class="pulse"></span><strong>30 minutes ahead.</strong><p>A little preparation.<br>A calmer day.</p></div>
<div class="profile"><div class="avatar">AA</div><div><strong>Prof. Dr. Asif Ahmad</strong><span>Personal workspace</span></div></div></aside>
<main><header class="topbar"><div class="mobile-brand">asif<span> · </span>schedule</div><div id="breadcrumb">Workspace / Overview</div><span class="timezone">◷ Pakistan time · UTC+5</span></header>
<div id="preview-banner" class="notice" hidden><strong>Private browser preview.</strong> Records here stay in this browser. Google sign-in, background reminders, and durable records run in the downloadable app. <a id="download-link" href="#connections">Get the runnable app →</a></div>
<div id="error-banner" class="notice error" hidden></div>
<section class="page" id="page-overview"><div class="page-head"><div><p class="eyebrow" id="today-date"></p><h1>A clear plan.<br><span>A calmer day.</span></h1><p class="muted">Your meetings, tasks, and reminders — in one place.</p></div><button class="primary" data-action="new">+ New event</button></div>
<div class="stats"><article><span>Today's schedule</span><strong id="stat-today">0</strong><small>meetings & tasks</small></article><article><span>Coming up this week</span><strong id="stat-week">0</strong><small>on your calendar</small></article><article><span>Invitations to review</span><strong id="stat-inbox">0</strong><small>Gmail</small></article><article class="reminder-stat"><span>Default reminder</span><strong>30 <em>min</em></strong><small>before every scheduled event</small></article></div>
<div class="overview-grid"><article class="panel next-panel"><div class="panel-title"><span class="eyebrow">UP NEXT</span><span class="badge">Personal calendar</span></div><div id="next-event"></div></article><article class="panel connection-summary"><p class="eyebrow">KEEP EVERYTHING CONNECTED</p><h3>Your schedule,<br>wherever it starts.</h3><div id="connection-mini"></div><a class="text-link" href="#connections">Manage connections ↗</a></article></div>
<article class="panel calendar-panel"><div class="section-head"><div><h2>Your week at a glance</h2><p class="muted" id="week-range"></p></div><div class="week-controls"><button class="icon-button" id="prev-week" aria-label="Previous week">‹</button><button class="light" id="this-week">Today</button><button class="icon-button" id="next-week" aria-label="Next week">›</button></div></div><div class="week-calendar" id="week-calendar"></div><div class="calendar-footer"><span><i class="dot meeting"></i>Meeting</span><span><i class="dot task"></i>Task</span><span><i class="dot appointment"></i>Appointment</span><span class="footer-right">Click a day to add an event</span></div></article></section>
<section class="page" id="page-agenda" hidden><div class="page-head compact"><div><p class="eyebrow">MAKE TIME FOR WHAT MATTERS</p><h1>My schedule</h1><p class="muted">Keep a complete record of your meetings and tasks.</p></div><button class="primary" data-action="new">+ New event</button></div><div class="filters"><input id="search" type="search" placeholder="Search your schedule…" aria-label="Search your schedule"><select id="agenda-filter" aria-label="Filter events"><option value="upcoming">Upcoming</option><option value="all">All records</option><option value="completed">Completed</option><option value="cancelled">Cancelled</option></select></div><div class="panel" id="agenda-list"></div></section>
<section class="page" id="page-inbox" hidden><div class="page-head compact"><div><p class="eyebrow">FROM CONVERSATION TO CALENDAR</p><h1>Invitation inbox</h1><p class="muted">Review suggested dates and times before adding anything.</p></div><button class="primary" id="paste-invite">+ Paste invitation</button></div><div class="notice">New matching emails appear here after connection. The date parser supports English; review other languages manually. Nothing is scheduled until you approve it.</div><div id="inbox-list"></div></section>
```

Appendix A - public/index.html (continued, 2 of 3) - lines 18-32 of 34

```
<section class="page" id="page-connections" hidden><div class="page-head compact"><div><p class="eyebrow">YOUR ACCOUNTS, YOUR CONTROL</p><h1>Connections</h1><p class="muted">Connect once. Keep your schedule together.</p></div><button class="light" id="refresh-connections">Refresh status</button></div>
<div class="connection-cards"><article class="panel integration"><div class="integration-logo gmail">M</div><div><h2>Gmail & Google Calendar</h2><p class="muted">Review invitations, sync calendar events, keep your Drive worksheet updated, and send reminders to yourself.</p></div><span id="google-status" class="badge">Not connected</span><div class="integration-body"><p id="google-account">asifahmad1@gmail.com</p><div id="google-setup"><label class="upload">1. Upload Google Desktop OAuth JSON<input id="google-json" type="file" accept=".json,application/json"></label><p class="small muted">Create a Desktop app client in Google Cloud; enable the Gmail, Google Calendar, Google Drive and Google Sheets APIs. Setup instructions are included with the download.</p></div><button class="primary" id="google-connect">2. Sign in with Google</button><button class="light" id="google-sync">Sync now</button><button class="text-button" id="google-disconnect">Disconnect</button><p class="small muted" id="sync-status">Not synced yet.</p></div></article>
</div>

<article class="panel setup-note"><h3>Google Drive worksheet</h3>
<p>Every schedule entry is automatically written as a row in the <strong>Asif Schedule Assistant</strong> worksheet in your Google Drive, with a dashboard of live counts.</p>
<p><span id="sheet-status" class="badge">Not created yet</span></p>
<p><a class="text-link" id="sheet-link" href="#" target="_blank" rel="noopener">Open worksheet ↗</a></p>
<div class="form-actions"><button class="light" id="sheet-sync">Sync worksheet now</button></div>
<p class="small muted" id="sheet-meta"></p>
</article><article class="panel setup-note"><h3>Background reminders need the runnable app</h3><p>Install the app on your computer and keep its reminder process running. The browser may be closed. If the computer sleeps or shuts down, Gmail reminders stop. Click “Save to Google” on a meeting to add Google's own 30-minute popup and email reminders, which do not depend on this process.</p><a class="text-link" id="app-download" href="#">Download app & setup guide ↗</a></article></section>
<section class="page" id="page-history" hidden><div class="page-head compact"><div><p class="eyebrow">A RECORD OF EVERY REMINDER</p><h1>Reminder history</h1><p class="muted">See accepted, unconfirmed, and dashboard reminders.</p></div></div><div class="panel" id="history-list"></div></section>
<section class="page" id="page-settings" hidden><div class="page-head compact"><div><p class="eyebrow">SET IT UP FOR YOUR DAY</p><h1>Settings</h1><p class="muted">All reminders are set for 30 minutes before your event.</p></div></div><form class="panel settings-form" id="settings-form"><label>Your Google email<input type="email" name="email" required></label><label>Time zone<input value="Asia/Karachi · Pakistan time (UTC+5)" disabled></label><label class="check"><input name="emailReminders" type="checkbox"> Email reminders to myself</label><label class="check"><input name="autoSync" type="checkbox"> Sync Google every five minutes</label><div class="form-actions"><button class="primary" type="submit">Save settings</button><button class="light" id="enable-notifications" type="button">Enable browser notifications</button><button class="light" id="export" type="button">Export records</button></div><p class="small muted">In the runnable app, records are stored on your computer and Google tokens are encrypted locally. Browser notifications need an open page; email reminders use the background process.</p></form></section>
<footer><span>Asif Schedule Assistant · Created by <strong>Dr. Asif Ahmad</strong></span><span class="footer-tag">Your time, thoughtfully organized.</span></footer></main>
<dialog id="event-dialog"><form id="event-form"><div class="dialog-head"><div><p class="eyebrow">A LITTLE PLANNING GOES A LONG WAY</p><h2 id="event-dialog-title">New event</h2></div><button type="button" class="icon-button" data-close>×</button></div><input name="id" type="hidden"><input name="inboxId" type="hidden"><label>Event title<input name="title" required maxlength="180" placeholder="e.g. Department research meeting"></label><div class="form-grid"><label>Type<select name="kind"><option value="meeting">Meeting</option><option value="task">Task</option><option value="appointment">Appointment</option></select></label><label>Date · Pakistan time<input name="date" type="date" required></label><label>Start time<input name="time" type="time" required></label><label>Duration<select name="duration"><option value="30">30 minutes</option><option value="60" selected>1 hour</option><option value="90">90 minutes</option><option value="120">2 hours</option><option value="240">4 hours</option></select></label></div><label>Location or meeting link<input name="location" placeholder="Office, room, or video meeting URL"></label><label>Notes<textarea name="notes" rows="3" placeholder="Anything to prepare beforehand"></textarea></label><div class="reminder-inline">◷ Reminder: <strong>30 minutes before</strong><span>via your enabled channels</span></div><p class="small muted" id="event-warning"></p><div class="form-actions"><button type="button" class="light" data-close>Cancel</button><button class="primary" type="submit">Save event</button></div></form></dialog>
```

Appendix A - public/index.html (continued, 3 of 3) - lines 33-34 of 34

```
33 <dialog id="invite-dialog"><form id="invite-form"><div class="dialog-head"><h2>Paste an invitation</h2
   ><button type="button" class="icon-button" data-close>×</button></div><label>Title<input name="title"
   placeholder="Invitation to review"></label><label>Source<select name="source"><option value="email">Em
   ail</option><option value="text">Other text</option></select></label><label>Message<textarea name="tex
   t" required rows="7" placeholder="Paste the email or message containing the meeting details…"></textar
   ea></label><p class="small muted">Suggested dates and times go to your inbox for review.</p><button ty
   pe="submit" class="primary">Review invitation</button></form></dialog>
34 <div id="toast" role="status" aria-live="polite"></div><script src="/app.js"></script></body></html>
```

Appendix A - public/app.js (continued, 1 of 5) - lines 1-33 of 123

```
const $=s=>document.querySelector(s),$$=s=>[...document.querySelectorAll(s)];
const preview=document.querySelector('meta[name="asif-mode"]')?.content==='preview';
const DEFAULTS={name:'Prof. Dr. Asif Ahmad',email:'asifahmad1@gmail.com',timezone:'Asia/Karachi',emailReminders:true,autoSync:true};
let state,weekOffset=0,page='overview',lastSeen=Date.now(),refreshing=false;
const esc=x=>String(x??'').replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]));
const fmt=(d,o)=>new Intl.DateTimeFormat('en-PK',{timeZone:'Asia/Karachi',...o}).format(new Date(d));
const dateKey=d=>new Date(new Date(d).getTime()+5*3600000).toISOString().slice(0,10);
const timeKey=d=>new Date(new Date(d).getTime()+5*3600000).toISOString().slice(11,16);
const stamp=d=>fmt(d,{dateStyle:'medium',timeStyle:'short'});
const time=d=>fmt(d,{hour:'numeric',minute:'2-digit',hour12:true});
const empty=(title,text,button='')=>`<div class="empty-state"><div class="empty-symbol" style="margin:auto">◎</div><h3>${esc(title)}</h3><p>${esc(text)}</p>${button}</div>`;
function toast(text){$('#toast').textContent=text;$('#toast').classList.add('show');setTimeout(()=>$('#toast').classList.remove('show'),4500);}
function previewState(){let saved;try{saved=JSON.parse(localStorage.getItem('asif-schedule-preview')||'{}');}catch{saved={};}return {settings:{...DEFAULTS,...saved.settings},events:saved.events||[],inbox:saved.inbox||[],logs:saved.logs||[],connections:{google:{configured:false,connected:false}},sync:{last:null,busy:false},mode:'preview'};}
function savePreview(){localStorage.setItem('asif-schedule-preview',JSON.stringify({settings:state.settings,events:state.events,inbox:state.inbox,logs:state.logs}));}
async function api(endpoint,data){
  if(preview){
    if(endpoint==='/api/state'){state=previewState();return state;}
    if(endpoint==='/api/events'){const e={...data,id:crypto.randomUUID(),status:'scheduled',source:'manual',created:new Date().toISOString()};state.events.push(e);if(data.inboxId)state.inbox=state.inbox.filter(x=>x.id!==data.inboxId);savePreview();return e;}
    if(endpoint==='/api/settings'){state.settings={...state.settings,...data};savePreview();return {ok:true};}
    const m=endpoint.match(/^\/api\/events\/([^/]+)(?:\/(.*))?$/);if(m){const e=state.events.find(e=>e.id===m[1]);if(!e)throw new Error('Event not found.');if(!m[2])Object.assign(e,data);else if(m[2]==='status')e.status=data.status;else throw new Error('Google sync needs the runnable app and Google sign-in.');savePreview();return e;}
    if(endpoint==='/api/inbox/dismiss'){state.inbox=state.inbox.filter(x=>x.id!==data.id);savePreview();return {ok:true};}
    if(endpoint==='/api/import-text'){state.inbox.unshift({id:crypto.randomUUID(),source:data.source,title:data.title||'Invitation to review',body:data.text,received:new Date().toISOString(),confidence:'Choose the date and time in the review form.'});savePreview();return {ok:true};}
    throw new Error('Open the runnable app on your computer to connect accounts. This page is a browser preview.');
  }
  const r=await fetch(endpoint,{method:data===undefined?'GET':'POST',headers:data===undefined?{}:{'Content-Type':'application/json','X-Asif-Csrf':state?.csrf||''},body:data===undefined?undefined:JSON.stringify(data)});
  const value=await r.json();if(!r.ok)throw new Error(value.error||'Could not complete the action.');return value;
}
async function refresh(){if(refreshing)return;refreshing=true;try{state=await api('/api/state');$('#error-banner').hidden=true;render();checkNotifications();}catch(e){$('#error-banner').textContent='The app is not responding. Start the reminder process and refresh this page. Existing records shown may be out of date.';$('#error-banner').hidden=false;}finally{refreshing=false;}}
function activeEvents(){return state.events.filter(e=>e.status==='scheduled'&&Date.parse(e.end)>Date.now()).sort((a,b)=>Date.parse(a.start)-Date.parse(b.start));}
function buttons(e){const done=e.status!=='scheduled';return `<div class="row-actions"><button class="light" data-edit="${esc(e.id)}">Edit</button>${!done?`<button class="light" data-google="${esc(e.id)}">${e.google_id?'Update Google':'Save to Google'}</button><button class="light" data-complete="${esc(e.id)}">Done</button><button class="text-button" data-cancel="${esc(e.id)}">Cancel</button>`:`<span class="badge">${esc(e.status)}</span>`}</div>`;}
function render(){
  $('#today-date').textContent=fmt(new Date(),{weekday:'long',month:'long',day:'numeric',year:'numeric'}).toUpperCase();
  const events=activeEvents(),today=dateKey(new Date());
```

```
  $('#stat-today').textContent=state.events.filter(e=>e.status==='scheduled'&&dateKey(e.start)===today).length;
  $('#stat-week').textContent=events.filter(e=>Date.parse(e.start)<Date.now()+7*86400000).length;
  $('#stat-inbox').textContent=state.inbox.length;$('#inbox-count').textContent=state.inbox.length;
  const e=events[0];$('#next-event').innerHTML=e?`<h3 class="next-title">${esc(e.title)}</h3><p class="next-time">${esc(stamp(e.start))} · ${Math.round((Date.parse(e.end)-Date.parse(e.start))/60000)} minutes</p>${e.location?`<p>⌖ ${esc(e.location)}</p>`:''}<div class="next-meta"><span class="badge">${esc(e.kind)}</span><span>◷ Reminder at ${esc(time(Date.parse(e.start)-30*60000))}</span></div>`:`<div class="next-empty"><div class="empty-symbol">◷</div><div><div class="empty-title">Make room for a good day.</div><p>Your next meeting will appear here.<br>Add an event to get started.</p></div></div><button class="light" data-action="new">+ Add your first event</button>`;
  const google=state.connections.google;
  $('#connection-mini').innerHTML=`<div class="connection-mini-row"><span class="mini-logo">M</span><span>Gmail & Calendar</span><span>${google.connected?'Connected':'Not connected'}</span></div><div class="connection-mini-row"><span class="mini-logo">▦</span><span>Drive worksheet</span><span>${state.sheet?'Synced':'Not created'}</span></div>`;
  renderWeek();renderAgenda();renderInbox();renderConnections();renderHistory();
  if(document.activeElement?.closest('#settings-form'))return;
  const f=$('#settings-form');f.elements.email.value=state.settings.email;for(const k of ['emailReminders','autoSync'])f.elements[k].checked=state.settings[k];
}
function renderWeek(){
  const d=new Date(dateKey(new Date())+'T12:00:00+05:00');const monday=(d.getUTCDay()+6)%7;d.setUTCDate(d.getUTCDate()-monday+weekOffset*7);
  const last=new Date(d.getTime()+6*86400000);$('#week-range').textContent=`${fmt(d,{month:'short',day:'numeric'})} – ${fmt(last,{month:'short',day:'numeric',year:'numeric'})}`;
  $('#week-calendar').innerHTML=Array.from({length:7},(_,i)=>{
    const day=new Date(d.getTime()+i*86400000),key=dateKey(day),events=state.events.filter(e=>e.status==='scheduled'&&dateKey(e.start)===key).sort((a,b)=>Date.parse(a.start)-Date.parse(b.start));
    return `<div class="day ${key===dateKey(new Date())?'today':''}" data-day="${key}"><span class="day-name">${fmt(day,{weekday:'short'}).toUpperCase()}</span><span class="day-number">${fmt(day,{day:'numeric'})}</span>${events.length?events.map(e=>`<div class="calendar-event ${esc(e.kind)}" data-edit="${esc(e.id)}"><small>${esc(time(e.start))}</small><strong>${esc(e.title)}</strong></div>`).join(''):'<p class="day-empty">+</p>'}</div>`;
  }).join('');
}
function renderAgenda(){
  const query=$('#search').value.toLowerCase(),filter=$('#agenda-filter').value;
  const events=state.events.filter(e=>(!query||(e.title+' '+e.location+' '+e.notes).toLowerCase().includes(query))&&(filter==='all'||(filter==='upcoming'?e.status==='scheduled'&&Date.parse(e.end)>=Date.now():e.status===filter))).sort((a,b)=>Date.parse(a.start)-Date.parse(b.start));
  let day='';$('#agenda-list').innerHTML=events.length?events.map(e=>{const key=dateKey(e.start),header=key!==day?`<div class="agenda-date-label">${esc(fmt(e.start,{weekday:'long',month:'short',day:'numeric',year:'numeric'}))}</div>`:'';day=key;return header+`<div class="agenda-row"><div class="agenda-time">${esc(time(e.start))}<small>${Math.round((Date.parse(e.end)-Date.parse(e.start))/60000)} min</small></div><div class="agenda-info"><strong>${esc(e.title)}</strong><span class="badge">${esc(e.kind)}</span> <span class="small muted">${esc(e.source==='google'?'Google Calendar':e.source)}</span>${e.location?`<p>${esc(e.location)}</p>`:''}<p>Reminder: ${esc(time(Date.parse(e.start)-1800000))} · Pakistan time</p></div>${buttons(e)}</div>`;}).join(''):empty('Nothing on this list yet.','Add a meeting, task, or appointment. Your records will stay here after the event is over.','<button class="primary" data-action="new">+ Add an event</button>');
}
function renderInbox(){
  $('#inbox-list').innerHTML=state.inbox.length?state.inbox.map(x=>`<article class="panel invite-card"><div class="section-head"><h3>${esc(x.title)}</h3><span class="badge">${esc(x.source)}</span></div><pre>${esc(x.body)}</pre><p class="suggestion">${x.suggested_start?esc(stamp(x.suggested_start))+' · ':''}${esc(x.confidence||'Review the date and time.')}</p><div class="invite-actions"><button class="primary" data-review="${esc(x.id)}">Review & add event</button><button class="light" data-dismiss="${esc(x.id)}">Dismiss</button></div></article>`).join(''):empty('A quieter inbox. A clearer calendar.','Connect Gmail to review invitations, or paste a message manually.');
}
function renderConnections(){
  const g=state.connections.google;
```

```
  $('#google-status').textContent=g.connected?'Connected':g.configured?'Ready to sign in':'Not connected';$('#google-status').className='badge '+(g.connected?'connected':g.configured?'pending':'');
  $('#google-account').textContent=g.email||state.settings.email;$('#google-setup').hidden=g.connected;
  $('#google-connect').hidden=g.connected;$('#google-connect').disabled=preview||!g.configured;$('#google-json').disabled=preview;
  $('#google-sync').hidden=!g.connected;$('#google-disconnect').hidden=!g.connected;
  $('#sync-status').textContent=state.sync.error||(state.sync.busy?'Syncing calendar and invitations…':state.sync.last?'Last synced '+stamp(state.sync.last):'Not synced yet.');
  const sh=state.sheet,ss=$('#sheet-status');
  if(ss){ss.textContent=!g.connected?'Needs Google sign-in':sh?(sh.error||'Ready'):'Not created yet';ss.className='badge '+(sh&&!sh.error&&g.connected?'connected':sh&&sh.error?'pending':'');}
  const sl=$('#sheet-link');if(sl){if(sh&&sh.url){sl.href=sh.url;sl.style.display='';}else sl.style.display='none';}
  const sm=$('#sheet-meta');if(sm)sm.textContent=sh&&sh.last?'Last synced '+stamp(sh.last)+(sh.busy?' · syncing…':''):'The worksheet is created in your Google Drive when you connect Google.';
}
function renderHistory(){
  $('#history-list').innerHTML=state.logs.length?state.logs.map(x=>`<div class="history-row"><div><strong>${esc(x.title||'Event reminder')}</strong><p>${esc(x.channel)} · ${esc(stamp(x.created))}</p><p>${esc(x.detail||'')}</p>${x.status==='uncertain'?`<button class="light" data-retry="${esc(x.id)}">Retry reminder</button>`:''}</div><span class="badge ${x.status==='sent'?'connected':'pending'}">${x.status==='sent'?'Accepted':esc(x.status)}</span></div>`).join(''):empty('Every reminder has a record.','Delivery records will appear when an event reaches its 30-minute reminder time.');
}
function showPage(){const hash=location.hash.slice(1);page=['overview','agenda','inbox','connections','history','settings'].includes(hash)?hash:'overview';$$('.page').forEach(x=>x.hidden=x.id!=='page-'+page);$$('nav a').forEach(x=>x.classList.toggle('active',x.dataset.page===page));$('#breadcrumb').textContent='Workspace / '+({agenda:'My schedule',inbox:'Invitation inbox',history:'Reminder history'}[page]||page[0].toUpperCase()+page.slice(1));}
function openEvent(event=null,day=null,inbox=null){
  const f=$('#event-form');f.reset();f.elements.id.value=event?.id||'';f.elements.inboxId.value=inbox?.id||'';
  const future=new Date(Date.now()+3600000),start=event?.start||inbox?.suggested_start||future.toISOString();
  f.elements.title.value=event?.title||inbox?.title||'';f.elements.date.value=day||dateKey(start);f.elements.time.value=timeKey(start);f.elements.kind.value=event?.kind||'meeting';f.elements.location.value=event?.location||'';f.elements.notes.value=event?.notes||inbox?.body||'';
  const duration=event?Math.max(1,Math.round((Date.parse(event.end)-Date.parse(event.start))/60000)):inbox?.suggested_end?Math.round((Date.parse(inbox.suggested_end)-Date.parse(start))/60000):60;
  if(![...f.elements.duration.options].some(x=>Number(x.value)===duration))f.elements.duration.add(new Option(duration+' minutes',duration));f.elements.duration.value=duration;
  $('#event-dialog-title').textContent=event?'Edit event':inbox?'Review invitation':'New event';$('#event-warning').textContent=inbox?'Confirm the suggested date, time, and details before saving.':event?.google_id?'Editing this event also updates Google Calendar when connected.':'';
  $('#event-dialog').showModal();
}
async function action(fn){try{await fn();await refresh();}catch(e){toast(e.message);}}
document.addEventListener('click',e=>{
  const t=e.target.closest('button,[data-edit],[data-day]');if(!t)return;
  if(t.hasAttribute('data-close'))return t.closest('dialog').close();
  if(t.dataset.action==='new')return openEvent();
  if(t.dataset.edit)return openEvent(state.events.find(x=>x.id===t.dataset.edit));
  if(t.dataset.day)return openEvent(null,t.dataset.day);
  if(t.dataset.review)return openEvent(null,null,state.inbox.find(x=>x.id===t.dataset.review));
  if(t.dataset.retry&&confirm('The previous delivery was not confirmed. Retrying may send a duplicate. Send again?'))return void action(async()=>{await api('/api/reminders/'+t.dataset.retry,{});toast('Retry processed. Check reminder history for the result.');});
  if(t.dataset.dismiss)return void action(async()=>{await api('/api/inbox/dismiss',{id:t.dataset.dismiss});toast('Invitation dismissed.');});
  if(t.dataset.google)return void action(async()=>{t.disabled=true;try{await api('/api/events/'+t.dataset.google+'/google',{});toast('Saved to Google with 30-minute popup and email reminders.');}finally{t.disabled=false;}});
```

Appendix A - public/app.js (continued, 4 of 5) - lines 96-122 of 123

```
  if(t.dataset.complete)return void action(async()=>{await api('/api/events/'+t.dataset.complete+'/status',{status:'completed'});toast('Marked completed. Your record is kept.');});
  if(t.dataset.cancel&&confirm('Cancel this event? If linked and Google is connected, it will also be cancelled in Google Calendar.'))return void action(async()=>{await api('/api/events/'+t.dataset.cancel+'/status',{status:'cancelled'});toast('Event cancelled. Your record is kept.');});
});
$('#event-form').addEventListener('submit',e=>{e.preventDefault();void action(async()=>{
  const f=e.target,b=Object.fromEntries(new FormData(f)),start=new Date(b.date+'T'+b.time+':00+05:00');if(!Number.isFinite(start.getTime()))throw new Error('Choose a valid date and time.');
  const payload={title:b.title,kind:b.kind,start:start.toISOString(),end:new Date(start.getTime()+Number(b.duration)*60000).toISOString(),location:b.location,notes:b.notes,inboxId:b.inboxId};
  const button=f.querySelector('button[type="submit"]');button.disabled=true;try{await api(b.id?'/api/events/'+b.id:'/api/events',payload);$('#event-dialog').close();toast('Event saved. Reminder set for 30 minutes before.');}finally{button.disabled=false;}
});});
$('#paste-invite').onclick=()=>{$('#invite-form').reset();$('#invite-dialog').showModal();};
$('#invite-form').onsubmit=e=>{e.preventDefault();void action(async()=>{await api('/api/import-text',Object.fromEntries(new FormData(e.target)));$('#invite-dialog').close();toast('Invitation added for review.');});};
$('#settings-form').onsubmit=e=>{e.preventDefault();const f=e.target;void action(async()=>{await api('/api/settings',{email:f.elements.email.value,emailReminders:f.elements.emailReminders.checked,autoSync:f.elements.autoSync.checked});toast('Settings saved.');});};
$('#google-json').onchange=e=>void action(async()=>{const file=e.target.files[0];if(!file)return;if(file.size>100000)throw new Error('Upload a Google OAuth JSON under 100 KB.');await api('/api/google/config',JSON.parse(await file.text()));toast('Google client configured. You can sign in now.');e.target.value='';});
$('#google-connect').onclick=()=>void action(async()=>{const r=await api('/api/google/connect',{});location.assign(r.url);});
$('#google-sync').onclick=()=>void action(async()=>{await api('/api/sync',{});toast('Sync started.');});
$('#sheet-sync').onclick=()=>void action(async()=>{await api('/api/sheet/sync',{});toast('Worksheet sync started.');});
$('#google-disconnect').onclick=()=>void action(async()=>{await api('/api/google/disconnect',{});toast('Google disconnected.');});
$('#refresh-connections').onclick=()=>void refresh();$('#search').oninput=renderAgenda;$('#agenda-filter').onchange=renderAgenda;
$('#prev-week').onclick=()=>{weekOffset--;renderWeek();};$('#next-week').onclick=()=>{weekOffset++;renderWeek();};$('#this-week').onclick=()=>{weekOffset=0;renderWeek();};
$('#enable-notifications').onclick=async()=>{if(!('Notification'in window))return toast('This browser does not support notifications.');const p=await Notification.requestPermission();toast(p==='granted'?'Browser notifications enabled. Keep this page open to receive them.':'Notifications were not enabled. You can change this in browser settings.');};
$('#export').onclick=()=>{if(!preview)return location.assign('/api/export');const blob=new Blob([JSON.stringify({exported:new Date().toISOString(),events:state.events,inbox:state.inbox,logs:state.logs},null,2)],{type:'application/json'}),a=document.createElement('a');a.href=URL.createObjectURL(blob);a.download='asif-schedule-preview-backup.json';a.click();setTimeout(()=>URL.revokeObjectURL(a.href),1000);};
function checkNotifications(){
  if(preview){for(const e of state.events.filter(e=>e.status==='scheduled'&&Date.parse(e.start)>Date.now()&&Date.parse(e.start)<=Date.now()+1800000)){if(state.logs.some(l=>l.event_id===e.id&&l.event_start===e.start))continue;state.logs.unshift({id:crypto.randomUUID(),event_id:e.id,event_start:e.start,title:e.title,channel:'dashboard',status:'sent',created:new Date().toISOString(),detail:'Browser preview reminder; email needs the runnable app.'});savePreview();}renderHistory();}
  for(const l of state.logs.filter(x=>x.channel==='dashboard'&&Date.parse(x.created)>lastSeen)){if('Notification'in window&&Notification.permission==='granted')new Notification('Upcoming: '+l.title,{body:'Starts at '+time(l.event_start)+' (Pakistan time)',tag:l.id});toast('Upcoming: '+l.title+' · '+time(l.event_start));}
  lastSeen=Date.now();
}
window.addEventListener('hashchange',showPage);showPage();
if(preview){$('#preview-banner').hidden=false;$('#download-link').href='/Asif-Schedule-Assistant.zip';$('#app-download').href='/Asif-Schedule-Assistant.zip';$('#google-connect').textContent='Sign-in available in runnable app';}else{$('#app-download').href='/setup';$('#app-download').textContent='Read setup guide ↗';}
```

Appendix A - public/app.js (continued, 5 of 5) - lines 123-123 of 123

```
123  void refresh();setInterval(()=>void refresh(),preview?15000:5000);
```

Appendix A - package.json - lines 1-18 of 18

```
{
  "name": "asif-schedule-assistant",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "Private Gmail, Google Calendar and Google Drive scheduling assistant with 30-minute reminders and an auto-updating worksheet.",
  "engines": {
    "node": ">=24"
  },
  "scripts": {
    "start": "node server.mjs",
    "test": "node --test test/*.test.mjs"
  },
  "dependencies": {
    "googleapis": "185.0.0",
    "chrono-node": "2.9.0"
  }
}
```

Appendix A - public/manifest.webmanifest - lines 1-1 of 1

```
1 {"background_color":"#f6f5f0","description":"Meetings, tasks and 30-minute reminders — created by Dr. Asif Ahmad.","display":"standalone","icons":[{"sizes":"192x192","src":"/icon-192.png","type":"image/png"},{"purpose":"any maskable","sizes":"512x512","src":"/icon-512.png","type":"image/png"}],"name":"Asif Schedule Assistant","orientation":"portrait","short_name":"Schedule","start_url":"/","theme_color":"#253842"}
```

Appendix A - Start-Windows.cmd - lines 1-26 of 26

```
@echo off
cd /d "%~dp0"
where node >nul 2>nul
if errorlevel 1 (
  echo Install Node.js 24 LTS from https://nodejs.org then run this file again.
  pause
  exit /b 1
)
node -e "if(!(Number(process.versions.node.split('.')[0])>=24))process.exit(1)"
if errorlevel 1 (
  echo Please install Node.js 24 or newer.
  pause
  exit /b 1
)
if not exist node_modules (
  call npm ci
  if errorlevel 1 (
    echo Dependency installation failed. Check your internet connection.
    pause
    exit /b 1
  )
)
start "" http://127.0.0.1:4318
echo Keep this window open for Gmail and WhatsApp reminders.
node server.mjs
pause
```

Appendix A - Start-Phone-Access.cmd - lines 1-30 of 30

```
@echo off
cd /d "%~dp0"
where node >nul 2>nul
if errorlevel 1 (
  echo Install Node.js 24 LTS from https://nodejs.org then run this file again.
  pause
  exit /b 1
)
node -e "if(!(Number(process.versions.node.split('.')[0])>=24))process.exit(1)"
if errorlevel 1 (
  echo Please install Node.js 24 or newer.
  pause
  exit /b 1
)
if not exist node_modules (
  call npm ci
  if errorlevel 1 (
    echo Dependency installation failed. Check your internet connection.
    pause
    exit /b 1
  )
)
echo.
echo Starting the app for PHONE access. Connect your phone to the SAME Wi-Fi as this laptop.
echo The phone address will be printed below. On your phone, open it in Chrome, then
echo use the browser menu -^> "Add to Home screen" to install the app.
echo.
set ASIF_BIND=lan
node server.mjs
pause
```

Appendix A - start.sh - lines 1-6 of 6

```
#!/usr/bin/env bash
set -e
cd "$(dirname "$0")"
node -e 'if(Number(process.versions.node.split(".")[0])<24) throw new Error("Install Node.js 24 or newer")'
if [ ! -d node_modules ]; then npm ci; fi
node server.mjs
```

APPENDIX B

Glossary and troubleshooting

Glossary

Term	Meaning
Node.js	The free program that runs the app's server file. Version 24 or newer is required; the LTS release from nodejs.org is the right one.
Localhost / 127.0.0.1	"This computer itself." An address that only your own machine can open - the basis of the app's privacy.
LAN	Your local network - the devices on your home or office Wi-Fi. Phone mode works on the LAN and nowhere else.
SQLite	A database that lives in a single file. Your records, settings and reminder log are all in it.
OAuth2 and PKCE	Google's sign-in method for apps. PKCE is an extra proof step that stops a stolen sign-in code from being reused by anyone else.
AES-256-GCM	Strong, standard encryption. Your Google tokens are sealed with it before they touch the disk.
CSRF token	A secret value the app checks on every change-making request, proving the request came from its own page.
PWA	Progressive Web App: a web page that can be installed like an app - home-screen icon, full-screen window - which is how the Schedule Assistant sits on your phone.
Debounce	Waiting a few seconds after the last change before acting, so ten quick edits cause one worksheet update instead of ten.
UTC and Asia/Karachi	Times are stored in UTC (the world reference time) and always shown in Pakistan time, so displays never drift.

Troubleshooting FAQ

The address http://127.0.0.1:4318 does not open

The app is not running. Start it again (Start-Windows.cmd, or bash start.sh) and leave its window open. If the start window shows an error about the port being in use, the app may already be running in another window - close the duplicates, or restart the computer and start it once.

Google sign-in fails or loops

Check the four APIs are enabled (Gmail, Calendar, Drive, Sheets), that the OAuth client type is **Desktop app**, and that the JSON you uploaded is that client's file. Sign in as asifahmad1@gmail.com. The consent click must be done by you, in the browser, once.

Reminders are not arriving

Confirm the app was running and the computer awake and online 30 minutes before the event, and that reminders are switched on in Settings. Then open Reminder history: an "uncertain" entry means the email could not be confirmed - use its Retry button. Events saved to Google also remind through Google itself.

The worksheet is not updating

Open Connections and check the Google status shows connected, then click Sync now. Remember the sync waits four seconds after your last change. If the workbook was deleted in Drive, the next sync creates a fresh one.

Invitations are missing from the inbox

The scanner runs every five minutes and reads recent Gmail messages. All-day events and messages with no English date expression appear without a firm suggestion - paste the text in manually from the inbox page and set the time yourself.

> **Rebuilding the app from this document.** Appendix A is complete: save each listing under its file name, keeping the `public/` files in a public folder, install Node.js 24, and start it as Section 5 describes. The only file not reprinted is `public/style.css`, which controls appearance only.

Schedule Assistant version 2.0 - designed and generated by Dr. Asif Ahmad. This documentation, including Appendix A, is the complete record of the app as built and tested (15 of 15 automated tests passing).